Lógica de Programação | Aula 10

Listas e Coleções em Python

João Quentino

11 de Março de 2026

Jovem Tech | Turma 04A - Manhã

Recapitulação

O que já aprendemos:

Estruturas de Controle:

- Condicionais (if/elif/else)
- Loop FOR
- Loop WHILE
- break e continue

Tipos de Dados Básicos:

- int, float
- str (strings)
- bool
- Variáveis simples

Hoje: Aprenderemos a armazenar múltiplos valores em uma única variável!

Agenda de Hoje

Introdução às Listas

Índices e Acesso

Fatiamento (Slicing)

Mutabilidade

Métodos de Lista

Operações com Listas

Exercícios Práticos

Projeto Integrador

Boas Práticas

Recapitulação

Introdução às Listas

Por Que Precisamos de Listas?

Problema sem listas:

Imagine armazenar as notas de 30 alunos:

- `nota1 = 8.5`
- `nota2 = 7.0`
- `nota3 = 9.5`
- ...
- `nota30 = 6.5`

Problemas:

- 30 variáveis diferentes!
- Impossível usar loops eficientemente
- Código gigantesco e repetitivo
- E se forem 100 alunos? 1000?

Solução: Listas!

Com listas, armazenamos tudo em uma variável:

```
notas = [8.5, 7.0, 9.5, 6.0, 8.0, 7.5, 9.0, ...]
```

Vantagens:

- Uma única variável para múltiplos valores
- Fácil de iterar com loops
- Crescimento dinâmico (adiciona/remove itens)
- Organização lógica dos dados

Lista = coleção ordenada de elementos

O que são Listas?

Definição:

Lista é uma estrutura de dados que armazena uma sequência ordenada de elementos.

Características:

- **Ordenada**: elementos têm posição definida
- **Indexada**: cada elemento tem um índice (0, 1, 2, ...)
- **Mutável**: pode ser modificada após criação
- **Heterogênea**: pode conter tipos diferentes
- **Dinâmica**: tamanho pode mudar

Em outras linguagens: Arrays, ArrayLists, Vectors

Criando Listas

Sintaxe: usar colchetes [ ]

```python
# Lista vazia
lista_vazia = []

# Lista de numeros
numeros = [1, 2, 3, 4, 5]

# Lista de strings
frutas = ["maca", "banana", "laranja"]

# Lista de tipos mistos
misturada = [10, "texto", 3.14, True]

# Lista de listas (matriz)
matriz = [[1, 2], [3, 4], [5, 6]]
```

Índices e Acesso

Conceito de Índice

Cada elemento da lista tem uma posição (índice):

	[0]	[1]	[2]	[3]	[4]
`numeros =`	10	20	30	40	50
	-5	-4	-3	-2	-1

Importante:

- Índices positivos: começam do 0
- Índices negativos: começam do -1 (do fim)
- Python é **zero-indexed** (primeiro elemento = índice 0)

Acessando Elementos

Use colchetes com o índice:

```
frutas = ["maca", "banana", "laranja", "uva", "manga"]

# Acesso por indice positivo
print(frutas[0])    # maca (primeiro)
print(frutas[2])    # laranja
print(frutas[4])    # manga (ultimo)

# Acesso por indice negativo
print(frutas[-1])   # manga (ultimo)
print(frutas[-2])   # uva (penultimo)
print(frutas[-5])   # maca (primeiro)
```

Erro comum: `IndexError` quando índice não existe

```
print(frutas[10])   # ERRO! Indice fora do intervalo
```

Tamanho da Lista

Use len() para saber quantos elementos:

```python
frutas = ["maca", "banana", "laranja"]
tamanho = len(frutas)
print(f"A lista tem {tamanho} elementos")  # 3

# Acessar ultimo elemento com len()
ultimo = frutas[len(frutas) - 1]
print(ultimo)  # laranja

# Ou simplesmente:
ultimo = frutas[-1]  # Mais facil!
```

Dica: Índices válidos vão de 0 até `len(lista) - 1`

Fatiamento (Slicing)

O que é Slicing?

Slicing permite extrair uma parte da lista:

Sintaxe: `lista[inicio:fim:passo]`

- `inicio`: índice inicial (incluído)
- `fim`: índice final (NÃO incluído)
- `passo`: intervalo entre elementos (opcional)

```python
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Elementos do indice 2 ate 5 (nao inclui 5)
parte = numeros[2:5]
print(parte)  # [2, 3, 4]
```

Slicing: Exemplos Básicos

```python
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Do inicio ate o indice 5 (nao inclui 5)
print(numeros[:5])      # [0, 1, 2, 3, 4]

# Do indice 5 ate o fim
print(numeros[5:])      # [5, 6, 7, 8, 9]

# Do indice 2 ate o 7
print(numeros[2:7])     # [2, 3, 4, 5, 6]

# Toda a lista
print(numeros[:])       # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Ultimos 3 elementos
print(numeros[-3:])     # [7, 8, 9]

# Primeiros 4 elementos
print(numeros[:4])      # [0, 1, 2, 3]
```

Slicing com Passo

O terceiro parâmetro define o intervalo:

```python
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Elementos pares (de 2 em 2)
print(numeros[::2])       # [0, 2, 4, 6, 8]

# Elementos impares
print(numeros[1::2])      # [1, 3, 5, 7, 9]

# Do indice 1 ao 8, de 2 em 2
print(numeros[1:8:2])     # [1, 3, 5, 7]

# Inverter a lista!
print(numeros[::-1])      # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Truque: `[::-1]` inverte qualquer sequência!

Visualizando Slicing

numeros[2:7:2]

Resultado: [2, 4, 6]

início=2, fim=7, passo=2 → pega índices 2, 4, 6

Mutabilidade

Listas são Mutáveis

Mutável = pode ser modificado após criação

```python
# Modificar um elemento
frutas = ["maca", "banana", "laranja"]
frutas[1] = "morango"
print(frutas)  # ["maca", "morango", "laranja"]

# Modificar multiplos elementos com slicing
numeros = [1, 2, 3, 4, 5]
numeros[1:4] = [20, 30, 40]
print(numeros)  # [1, 20, 30, 40, 5]
```

Comparação:

- Mutáveis: listas, dicionários, sets
- Imutáveis: strings, tuplas, números

Strings vs Listas

Strings são imutáveis, listas são mutáveis:

String (imutável):

```python
texto = "Python"
# ERRO! Nao pode modificar
# texto[0] = "J"

# Precisa criar nova string
texto = "J" + texto[1:]
print(texto)  # Jython
```

Lista (mutável):

```python
letras = ["P","y","t","h","o","n"]
# Pode modificar!
letras[0] = "J"
print(letras)
# ["J","y","t","h","o","n"]
```

Métodos de Lista

append() - Adicionar ao Final

Adiciona um elemento no final da lista:

```python
frutas = ["maca", "banana"]
print(frutas)  # ["maca", "banana"]

frutas.append("laranja")
print(frutas)  # ["maca", "banana", "laranja"]

frutas.append("uva")
print(frutas)  # ["maca", "banana", "laranja", "uva"]
```

Uso comum: Construir listas dinamicamente

```python
notas = []
for i in range(5):
    nota = float(input(f"Nota {i+1}: "))
    notas.append(nota)
```

pop() - Remover e Retornar

Remove e retorna um elemento (por padrão, o último):

```python
frutas = ["maca", "banana", "laranja", "uva"]

# Remove o ultimo
ultimo = frutas.pop()
print(ultimo)  # uva
print(frutas)  # ["maca", "banana", "laranja"]

# Remove por indice
fruta = frutas.pop(1)
print(fruta)   # banana
print(frutas)  # ["maca", "laranja"]

# Remove o primeiro
primeiro = frutas.pop(0)
print(primeiro)  # maca
print(frutas)    # ["laranja"]
```

sort() - Ordenar

Ordena a lista in-place (modifica a própria lista):

```python
# Ordenar numeros
numeros = [5, 2, 8, 1, 9, 3]
numeros.sort()
print(numeros)  # [1, 2, 3, 5, 8, 9]

# Ordenar strings (alfabetica)
frutas = ["laranja", "banana", "uva", "maca"]
frutas.sort()
print(frutas)  # ["banana", "laranja", "maca", "uva"]

# Ordem decrescente
numeros.sort(reverse=True)
print(numeros)  # [9, 8, 5, 3, 2, 1]
```

Nota: `sort()` modifica a lista original, não retorna nova

Outros Métodos Úteis

```python
numeros = [1, 2, 3, 2, 4, 2, 5]

# insert(indice, valor) - insere em posicao especifica
numeros.insert(0, 0)  # Insere 0 no inicio
print(numeros)  # [0, 1, 2, 3, 2, 4, 2, 5]
# remove(valor) - remove primeira ocorrencia
numeros.remove(2)  # Remove primeiro 2
print(numeros)  # [0, 1, 3, 2, 4, 2, 5]
# count(valor) - conta ocorrencias
qtd = numeros.count(2)
print(qtd)  # 2
# index(valor) - retorna indice da primeira ocorrencia
indice = numeros.index(4)
print(indice)  # 4
# reverse() - inverte a lista
numeros.reverse()
print(numeros)  # [5, 2, 4, 2, 3, 1, 0]
```

clear() e copy()

```python
# clear() - remove todos os elementos
frutas = ["maca", "banana", "laranja"]
frutas.clear()
print(frutas)  # []

# copy() - cria uma copia da lista
original = [1, 2, 3, 4, 5]
copia = original.copy()

copia.append(6)
print(original)  # [1, 2, 3, 4, 5]
print(copia)     # [1, 2, 3, 4, 5, 6]

# CUIDADO! Sem copy(), ambas apontam para mesma lista
lista1 = [1, 2, 3]
lista2 = lista1  # Nao e copia!
lista2.append(4)
print(lista1)  # [1, 2, 3, 4] - Modificou ambas!
```

Operações com Listas

Concatenação e Repetição

Operador +: concatenar listas

```
lista1 = [1, 2, 3]
lista2 = [4, 5, 6]
resultado = lista1 + lista2
print(resultado)  # [1, 2, 3, 4, 5, 6]
```

Operador *: repetir lista

```
zeros = [0] * 5
print(zeros)  # [0, 0, 0, 0, 0]

padrao = [1, 2] * 3
print(padrao)  # [1, 2, 1, 2, 1, 2]
```

Operador in

Verificar se elemento está na lista:

```
frutas = ["maca", "banana", "laranja"]

# Verificar presenca
if "banana" in frutas:
    print("Temos banana!")

if "uva" not in frutas:
    print("Nao temos uva")

# Exemplo pratico: validacao
cores_validas = ["vermelho", "azul", "verde", "amarelo"]
cor = input("Escolha uma cor: ")

if cor in cores_validas:
    print("Cor valida!")
else:
    print("Cor invalida!")
```

Iterando sobre Listas

Método 1: Iterar sobre valores

```python
frutas = ["maca", "banana", "laranja"]

for fruta in frutas:
    print(f"Eu gosto de {fruta}")
```

Método 2: Iterar com índices

```python
for i in range(len(frutas)):
    print(f"{i}: {frutas[i]}")
```

Método 3: enumerate() - índice e valor

```python
for i, fruta in enumerate(frutas):
    print(f"{i}: {fruta}")
```

List Comprehension (Preview)

Forma concisa de criar listas:

```python
# Forma tradicional
quadrados = []
for i in range(10):
    quadrados.append(i ** 2)
print(quadrados)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# List comprehension (mais avancado - opcional)
quadrados = [i ** 2 for i in range(10)]
print(quadrados)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Com condicao
pares = [i for i in range(20) if i % 2 == 0]
print(pares)  # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

Nota: List comprehension é opcional - veremos depois!

Exercícios Práticos

Exercício 1: Criar e Manipular

Desafio (10 min):

Crie um programa que:

1. Peça 5 números ao usuário
2. Armazene em uma lista
3. Exiba a lista completa
4. Exiba o maior número
5. Exiba o menor número
6. Exiba a média
7. Exiba a lista ordenada

Dicas:

- Use `append()` para adicionar
- Use `max(lista)` e `min(lista)`
- Use `sum(lista) / len(lista)` para média

Exercício 2: Lista de Compras

Desafio (15 min):

Crie um programa de lista de compras que:

1. Mostre menu:
   - 1 - Adicionar item
   - 2 - Remover item
   - 3 - Ver lista
   - 0 - Sair
2. Adicione itens com `append()`
3. Remova itens com `remove()`
4. Mostre a lista completa
5. Repita até escolher sair

Extra: Verifique se item existe antes de remover!

Exercício 3: Inverter Lista

Desafio (10 min):

Inverta uma lista SEM usar `reverse()` ou slicing:

```
numeros = [1, 2, 3, 4, 5]

# Complete usando loop e indices
invertida = []

# Seu codigo aqui...

print(invertida)  # [5, 4, 3, 2, 1]
```

Dicas:

- Use loop de trás para frente
- Ou use `pop()` para remover do fim e `append()` na nova lista

Exercício 4: Remover Duplicatas

Desafio (15 min):

Remova elementos duplicados de uma lista:

```python
numeros = [1, 2, 3, 2, 4, 1, 5, 3, 6, 4]

# Criar nova lista sem duplicatas
sem_duplicatas = []

# Seu codigo aqui...

print(sem_duplicatas)  # [1, 2, 3, 4, 5, 6]
```

Dica: Para cada número, verifique se já está na nova lista antes de adicionar

Exercício 5: Separar Pares e Ímpares

Desafio (15 min):

Separe uma lista em duas: pares e ímpares

```python
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

pares = []
impares = []

# Seu codigo aqui...

print(f"Pares: {pares}")        # [2, 4, 6, 8, 10]
print(f"Impares: {impares}")    # [1, 3, 5, 7, 9]
```

Dica: Use `if numero % 2 == 0` para verificar se é par

Exercício 6: Busca Linear

Desafio (15 min):

Encontre a posição de um elemento na lista (sem usar `index()`):

```python
numeros = [10, 25, 30, 15, 20, 35, 40]
procurado = 20

# Buscar posicao
posicao = -1  # -1 indica nao encontrado

# Seu codigo aqui...

if posicao != -1:
    print(f"Encontrado na posicao {posicao}")
else:
    print("Nao encontrado")
```

Projeto Integrador

Projeto: Sistema de Notas

Crie um sistema completo de gerenciamento de notas.

Funcionalidades:

1. Menu interativo com opções:
    - Adicionar nota
    - Remover nota (por índice)
    - Ver todas as notas
    - Calcular média
    - Ver maior e menor nota
    - Ver notas ordenadas
    - Ver quantas notas acima da média
    - Sair
2. Validações:
    - Notas devem estar entre 0 e 10
    - Não permitir remover de lista vazia
3. Interface clara e organizada

Projeto: Solução Parcial

```python
notas = []

while True:
    print("\n=== SISTEMA DE NOTAS ===")
    print("1 - Adicionar nota")
    print("2 - Remover nota")
    print("3 - Ver notas")
    print("4 - Calcular media")
    print("5 - Maior e menor")
    print("0 - Sair")

    opcao = input("Escolha: ")

    if opcao == "1":
        nota = float(input("Digite a nota: "))
        if 0 <= nota <= 10:
            notas.append(nota)
            print("Nota adicionada!")
        else:
            print("Nota invalida! Deve ser 0-10.")
```

Boas Práticas

Boas Práticas com Listas

Dicas importantes:

1. **Nomes descritivos**
   - `notas`, `alunos`, `produtos`
   - Evite `lista1`, `l`, `x`
2. **Verifique limites**
   - Use `if i < len(lista)` antes de acessar
   - Ou use `try/except IndexError`
3. **Cuidado com referências**
   - Use `.copy()` quando precisar de cópia
   - Atribuição simples cria referência, não cópia!
4. **Escolha o método certo**
   - `append()` vs `insert()`
   - `remove()` vs `pop()`
   - `sort()` vs `sorted()`

sort() vs sorted()

Qual usar?

sort() - in-place:

```
numeros = [3, 1, 4, 2]
numeros.sort()
print(numeros)
# [1, 2, 3, 4]

# Modifica original
# Nao retorna nada
```

Use quando quiser modificar a lista

sorted() - nova lista:

```
numeros = [3, 1, 4, 2]
ordenada = sorted(numeros)
print(numeros)
# [3, 1, 4, 2]
print(ordenada)
# [1, 2, 3, 4]
```

Use quando precisar da original

Recapitulação

O que Aprendemos Hoje

Listas em Python:

1. **Conceito**: Coleção ordenada e mutável
2. **Índices**: Positivos (0+) e negativos (-1-)
3. **Slicing**: [inicio:fim:passo]
4. **Mutabilidade**: Pode modificar após criar
5. **append()**: Adicionar ao final
6. **pop()**: Remover e retornar
7. **sort()**: Ordenar in-place
8. **Operações**: +, *, in, len()

Você agora domina estruturas de dados básicas!

Tabela de Métodos

Método	Descrição
`append(x)`	Adiciona x ao final
`insert(i, x)`	Insere x na posição i
`remove(x)`	Remove primeira ocorrência de x
`pop(i)`	Remove e retorna item na posição i
`clear()`	Remove todos os elementos
`index(x)`	Retorna índice de x
`count(x)`	Conta ocorrências de x
`sort()`	Ordena a lista
`reverse()`	Inverte a lista
`copy()`	Retorna cópia da lista

Próximos Passos

O que vem pela frente:

- **Tuplas**: Listas imutáveis
- **Dicionários**: Pares chave-valor
- **Sets**: Conjuntos sem duplicatas
- **Funções**: Organizar código em blocos reutilizáveis
- **Módulos**: Importar e usar bibliotecas

Continue praticando listas! São fundamentais para tudo.

Desafio para Casa

Projetos para praticar:

1. **Jogo da Memória de Números:**
   - Mostre 5 números por 2 segundos
   - Limpe a tela
   - Peça ao usuário para reproduzir
2. **Analisador de Texto:**
   - Peça uma frase
   - Conte palavras, vogais, consoantes
   - Encontre a palavra mais longa
3. **Simulador de Baralho:**
   - Crie lista com 52 cartas
   - Embaralhe (use random)
   - Distribua 5 cartas para jogador

Dúvidas e Encerramento

Momento para perguntas!

Contato:

joao.quentino@dcomp.ufs.br

Obrigado!

Próxima aula: Tuplas e Dicionários!